ASSOCIATION DES PROFESSIONNELS DE LA SÉCURITÉ DE L'INFORMATION DU NIGER (APSI-NE)

Formation IA et cybersécurité — Projet de fin de formation, session 2026

Réalisé dans le cadre de la filière Génie logiciel de l'EMIG, Niamey, Niger

PROJET FINAL APSI-NE — SUJET A

Conception et évaluation d'un système intelligent de détection d'intrusions réseau par Machine Learning

Application au dataset UNSW-NB15

Organisme de formation	APSI-NE — Formation IA et cybersécurité, session 2026
Présenté par	Seydou Dangande Abdoul Aziz
Établissement d'origine	École des Mines, de l'Industrie et de la Géologie (EMIG)
Filière	Génie logiciel
Année	2026

Rapport technique de synthèse — expérimentation sur jeu de données public, ne constituant pas une validation de déploiement en production.

1. Résumé exécutif

Ce projet évalue l'apport du Machine Learning (apprentissage automatique) à la détection d'intrusions réseau, c'est-à-dire à la distinction entre le trafic normal et le trafic associé à une attaque. L'expérimentation s'appuie sur le dataset public UNSW-NB15 [1] et compare deux approches : **Random Forest**, un classifieur supervisé entraîné sur des exemples étiquetés [3], et **Isolation Forest**, une méthode de détection d'anomalies entraînée uniquement sur du trafic normal [4].

Sur le jeu de test officiel (82 332 observations), Random Forest obtient une accuracy de 89,29 %, une précision de 85,19 %, un rappel de 97,51 % et un F1-score de 90,93 % sur la classe des attaques, ainsi qu'un ROC-AUC de 98,07 %. Isolation Forest, avec son seuil automatique, obtient une accuracy de 59,11 % et un rappel des attaques de 33,08 %. Random Forest est donc nettement la meilleure des configurations testées. Son analyse détaillée montre toutefois un coût : 7 687 fausses alertes et 1 128 attaques non détectées sur le jeu de test.

Le rapport complète ces résultats par une analyse des erreurs, l'examen des variables les plus importantes, une simulation synthétique de dérive des données (effet limité sur les indicateurs dans ce scénario précis), des principes de gouvernance fondés sur la supervision humaine, et des recommandations réalistes.

> **Portée des résultats.** Ils sont liés au dataset UNSW-NB15 et au protocole utilisé. Ils ne garantissent pas les mêmes performances sur un réseau réel, ne prouvent pas la détection d'attaques inconnues et ne valident pas un déploiement en production.

Modèle	Accuracy	Précision attaques	Rappel attaques	F1 attaques	ROC-AUC
Random Forest	89,29 %	85,19 %	97,51 %	90,93 %	98,07 %
Isolation Forest	59,11 %	81,82 %	33,08 %	47,12 %	77,42 %

Tableau 1 — Synthèse des performances sur le jeu de test (82 332 observations).

2. Introduction et contexte

Les réseaux informatiques sont exposés à des activités malveillantes variées. Un **système de détection d'intrusions** (IDS) surveille le trafic et signale les comportements suspects. Les approches à base de signatures ou de règles prédéfinies sont efficaces contre des motifs connus, mais peuvent être limitées lorsqu'un comportement ne ressemble à rien de déjà répertorié.

Le Machine Learning apprend des régularités à partir d'exemples de trafic. Deux logiques s'opposent. L'approche **supervisée** apprend à partir de trafic déjà étiqueté « normal » ou « attaque » : c'est un agent de sécurité formé avec des portraits de cambrioleurs connus. La **détection d'anomalies** apprend uniquement à quoi ressemble le trafic normal et signale ce qui s'en écarte : c'est un gardien qui connaît tous les habitués d'un immeuble et remarque tout visage inconnu.

Dans un **centre opérationnel de sécurité** (SOC, *Security Operations Center*), les alertes produites par de tels modèles sont examinées par des analystes. La qualité d'un modèle ne se mesure donc pas seulement à son taux de bonnes prédictions, mais aussi à la charge de travail et aux risques que ses erreurs engendrent. Cette étude est une expérimentation sur données préparées, et non un capteur réseau opérationnel.

3. Problématique et objectifs

Problématique. Dans quelle mesure des modèles de Machine Learning supervisé et de détection d'anomalies permettent-ils d'identifier les activités d'attaque dans UNSW-NB15, et quelles différences de performance observe-t-on entre ces deux approches ?

Objectifs.

- Explorer le dataset et caractériser les données disponibles, puis préparer variables explicatives et variable cible.
- Construire un classifieur supervisé Random Forest et expérimenter Isolation Forest sur le trafic normal uniquement.
- Comparer les résultats, analyser les erreurs (fausses alertes, attaques manquées) et interpréter les variables importantes.
- Mesurer la sensibilité du meilleur modèle à une dérive de données simulée, en précisant les limites de cette simulation.
- Formuler des principes de gouvernance, de surveillance et des recommandations d'amélioration.

4. Présentation du dataset UNSW-NB15

UNSW-NB15 a été produit par le Cyber Range Lab de l'UNSW Canberra : l'outil IXIA PerfectStorm y a généré un mélange d'activités normales modernes et de comportements d'attaque synthétiques contemporains, et 100 Go de trafic brut ont été capturés avec tcpdump [1], [2]. Le dataset comporte neuf familles d'attaques : Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode et Worms [2]. Les *Fuzzers* envoient des données aléatoires ou malformées pour provoquer des dysfonctionnements ; les *Exploits* tirent parti de vulnérabilités connues.

Le projet utilise la partition officielle en deux fichiers [2] : un jeu d'entraînement de 175 341 observations et un jeu de test de 82 332 observations, tous deux de 45 colonnes. Chaque observation décrit une connexion réseau (flux) par des variables telles que la durée, le protocole, le service, l'état, les nombres de paquets et d'octets, les débits ou les valeurs TTL, auxquelles s'ajoutent la cible binaire `label` (0 = normal, 1 = attaque) et la catégorie détaillée `attack_cat`.

Jeu	Observations	Normal (0)	Attaque (1)
Entraînement	175 341	56 000 (31,94 %)	119 341 (68,06 %)
Test	82 332	37 000 (44,94 %)	45 332 (55,06 %)

Tableau 2 — Taille et répartition des classes des deux fichiers utilisés.

Les contrôles de qualité n'ont révélé ni valeur manquante dans le jeu d'entraînement, ni ligne dupliquée, ni valeur infinie dans les deux jeux. La répartition des classes diffère toutefois entre entraînement (68,06 % d'attaques) et test (55,06 %). Cette différence invite à la prudence : la précision et l'accuracy dépendent de la proportion d'attaques, d'où l'usage complémentaire du rappel, du F1-score et des matrices de confusion.

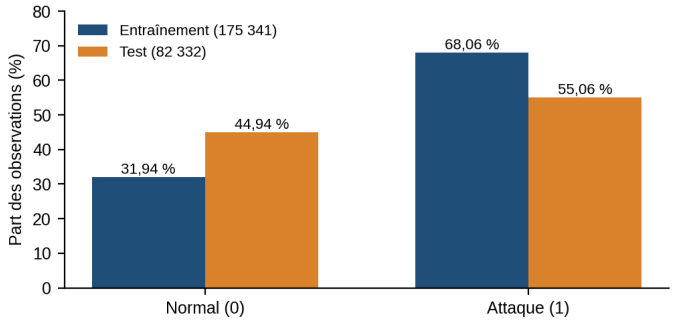


Figure 1 — Répartition des classes dans les jeux d'entraînement et de test.

5. Prétraitement et préparation des données

Trois colonnes ne sont pas utilisées comme variables d'entrée : `id` (simple identifiant), `label` (la cible) et `attack_cat`. Cette dernière est exclue car fournir la catégorie détaillée de l'attaque reviendrait à donner indirectement la réponse à un modèle dont l'objectif est une classification binaire. Il reste **42 variables explicatives** : 39 numériques et 3 catégorielles (`proto`, `service` et `state`, qui comptent respectivement 133, 13 et 9 modalités dans le jeu d'entraînement).

Les variables catégorielles sont transformées par *One-Hot Encoding* : chaque modalité devient une colonne valant 0 ou 1, et les modalités inconnues rencontrées au test sont ignorées. Pour Random Forest, les variables numériques sont conservées telles quelles ; pour Isolation Forest, elles sont standardisées (*StandardScaler* : centrage et réduction). Toutes ces transformations sont intégrées dans des *pipelines* ajustés sur les seules données d'entraînement, ce qui évite la fuite de données (*data leakage*) : le jeu de test n'influence ni le prétraitement ni l'apprentissage.

6. Présentation des modèles

6.1 Random Forest — apprentissage supervisé

Random Forest combine les votes de nombreux arbres de décision construits sur des échantillons différents des données [3] : un comité d'experts dont la décision collective est plus stable que celle d'un expert isolé. Le modèle utilisé (*RandomForestClassifier* de scikit-learn [5], [6]) compte 100 arbres, une graine aléatoire fixée à 42 et une pondération équilibrée des classes (*class_weight="balanced"*) qui compense le déséquilibre entre classes. Les autres hyperparamètres gardent leurs valeurs par défaut : aucun réglage n'a été effectué. La prédiction est la classe la plus probable, soit un seuil de 0,5 sur la probabilité d'attaque.

6.2 Isolation Forest — détection d'anomalies

Isolation Forest isole les observations par des coupures aléatoires successives : un point atypique est séparé du reste avec peu de coupures, un point banal en demande beaucoup [4], [7]. Le modèle (*IsolationForest* de scikit-learn) est entraîné sur les seules 56 000 observations normales du jeu d'entraînement, avec 100 arbres, une graine fixée à 42 et *contamination="auto"*, valeur pour laquelle le seuil est déterminé comme dans l'article original [7]. Une observation jugée anormale est convertie en classe « attaque », les autres en classe « normal ». Le ROC-AUC est calculé à partir du score d'anomalie. Ce modèle ne voit aucune attaque pendant l'apprentissage.

7. Méthodologie d'évaluation

Les modèles sont entraînés sur le fichier d'entraînement puis évalués une seule fois sur le fichier de test officiel. La classe « attaque » est la classe positive. Dans une perspective SOC, un **vrai positif** (VP) est une attaque correctement signalée, un **faux positif** (FP) une fausse alerte sur du trafic normal, un **faux négatif** (FN) une attaque non détectée et un **vrai négatif** (VN) du trafic normal correctement ignoré.

Indicateur	Définition	Lecture pour un SOC
Accuracy	Part des prédictions correctes	Vision globale, sensible à la proportion des classes
Précision (attaques)	VP / (VP + FP)	Part des alertes qui sont de vraies attaques
Rappel (attaques)	VP / (VP + FN)	Part des attaques réellement détectées
F1-score	Moyenne harmonique précision–rappel	Compromis entre fausses alertes et attaques manquées
ROC-AUC	Probabilité qu'une attaque tirée au hasard reçoive un score supérieur à celui d'un trafic normal tiré au hasard [9]	Qualité du classement des observations, indépendamment du seuil

Tableau 3 — Indicateurs d'évaluation utilisés.

Aucun réglage de seuil n'est effectué à partir du jeu de test. Le protocole repose sur une exécution unique avec graine fixée (pas de validation croisée ni de jeu de validation) ; les résultats n'ont donc pas d'intervalle de confiance. L'environnement est décrit dans le fichier *requirements.txt* du projet (scikit-learn 1.9.1, pandas 3.1.0, NumPy 2.5.4, Matplotlib 3.11.2) [5], [10].

8. Résultats expérimentaux et comparaison

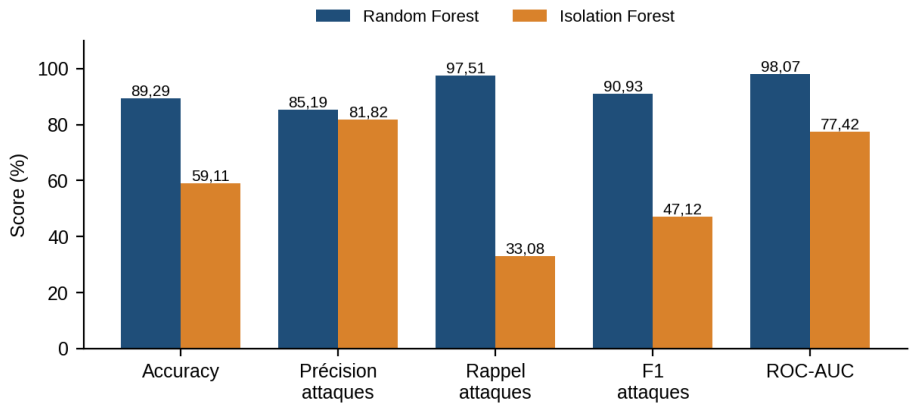


Figure 2 — Comparaison des indicateurs de Random Forest et d'Isolation Forest sur le jeu de test.

Modèle	Classe	Précision	Rappel	F1-score	Effectif
Random Forest	Normal	96,29 %	79,22 %	86,93 %	37 000
Random Forest	Attaque	85,19 %	97,51 %	90,93 %	45 332
Isolation Forest	Normal	52,60 %	90,99 %	66,67 %	37 000
Isolation Forest	Attaque	81,82 %	33,08 %	47,12 %	45 332

Tableau 4 — Indicateurs par classe (effectif = nombre d'observations réelles de la classe dans le test).

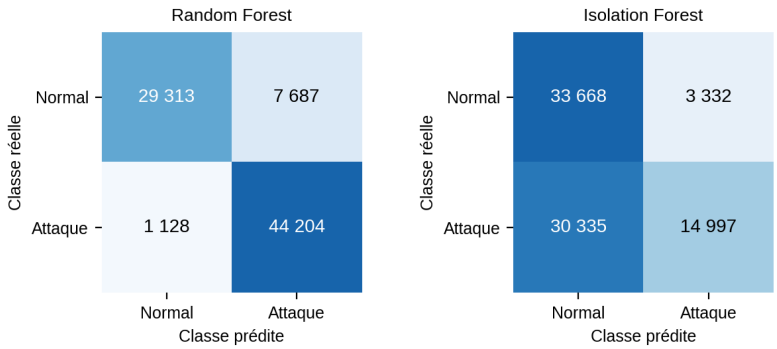


Figure 3 — Matrices de confusion sur le jeu de test (lignes : classe réelle ; colonnes : classe prédite).

Random Forest détecte 44 204 des 45 332 attaques (rappel de 97,51 %) et son ROC-AUC de 98,07 % indique un très bon classement des observations. Sa contrepartie est un rappel de 79,22 % sur le trafic normal : 7 687 observations normales sont classées comme attaques. **Isolation Forest** présente le profil inverse : 90,99 % du trafic normal est reconnu (33 668 VN, pour 3 332 FP), mais seules 14 997 attaques sont détectées et 30 335 sont classées à tort comme normales. Ce comportement est cohérent avec sa conception : n'ayant appris que le trafic normal, il ne signale que ce qui s'en écarte nettement.

Dans les configurations testées, Random Forest domine sur tous les indicateurs globaux et sur la classe des attaques, l'écart de précision étant toutefois modeste (85,19 % contre 81,82 %) ; Isolation Forest ne le devance que sur la reconnaissance du trafic normal (rappel de 90,99 % contre 79,22 %). Ce classement ne vaut que pour ces configurations : le ROC-AUC de 77,42 % d'Isolation Forest montre que son score contient une information de séparation, et son faible rappel est lié à son seuil automatique. Le réglage de ce seuil sur un jeu de validation indépendant n'a pas été étudié et son effet ne peut pas être chiffré ici.

9. Analyse des erreurs et interprétabilité

9.1 Faux positifs et faux négatifs de Random Forest

La matrice de confusion de Random Forest comprend 29 313 VN, 7 687 FP, 1 128 FN et 44 204 VP. Le taux de faux positifs est de 20,78 % du trafic normal et le taux de faux négatifs de 2,49 % des attaques. Parmi les 51 891 alertes émises (VP + FP), 14,81 % sont de fausses alertes, ce qui correspond à la précision de 85,19 %.

Impact sur un SOC. Un faux positif consomme du temps d'analyse ; en grand nombre, il provoque une fatigue d'alerte qui peut conduire à négliger de vraies alertes, et il peut perturber une activité légitime si une action automatique est déclenchée. Un faux négatif est plus grave par nature : une intrusion passe inaperçue et peut progresser (compromission d'autres machines, exfiltration de données) avant d'être découverte. Les deux coûts sont asymétriques, mais leur valeur dépend de l'organisation ; elle n'a pas été quantifiée dans ce projet. Pour Isolation Forest, les erreurs sont plutôt des attaques manquées (30 335 FN, soit 66,92 % des attaques) que des fausses alertes (3 332 FP, soit 9,01 % du trafic normal) : sa configuration actuelle laisserait passer la majorité des attaques de ce jeu de test.

Le notebook décrit la répartition des erreurs de Random Forest (Tableau 5). Il s'agit d'une analyse **descriptive** : une fréquence élevée ne prouve pas que la variable cause l'erreur. À titre de repère, le service « - » (non identifié) représente à lui seul 53,7 % du jeu d'entraînement ; sa présence majoritaire parmi les erreurs n'est donc pas surprenante.

Critère	Faux positifs (7 687)	Faux négatifs (1 128)
Protocole	tcp : 5 333 ; udp : 2 354	tcp : 859 ; udp : 269
Service	« - » : 6 506 ; http : 942 ; ftp : 236 ; autres : 3	« - » : 1 002 ; ftp : 98 ; http : 28
État de la connexion	FIN : 5 328 ; INT : 2 339 ; autres : 20	FIN : 859 ; INT : 269
Catégorie d'attaque	— (trafic normal)	Fuzzers : 1 031 ; Exploits : 77 ; autres : 20 (DoS 8, Shellcode 5, Reconnaissance 4, Generic 2, Analysis 1)

Tableau 5 — Répartition descriptive des erreurs de Random Forest (valeurs issues du notebook).

Deux constats ressortent. D'abord, les attaques manquées sont dans leur très grande majorité des *Fuzzers* (1 031 sur 1 128, soit 91,4 %) ; le rappel par catégorie n'a pas été calculé, faute des effectifs par catégorie dans les résultats conservés, et l'on ne peut donc pas affirmer que cette famille est intrinsèquement plus difficile. Ensuite, les scores

d’attaque observés en exemple se situent près de la frontière de décision pour plusieurs faux négatifs : les dix faux négatifs les plus proches du seuil ont tous un score de 0,50 (à deux décimales). Les cinq faux positifs affichés, tous des flux *udp* à l’état INT, ont en revanche des scores de 0,55 à 0,83, donc éloignés du seuil. Un ajustement du seuil pourrait ainsi récupérer certains faux négatifs, mais au prix d’autres fausses alertes, et ne suffirait pas à corriger tous les faux positifs : il devra être étudié sur un jeu de validation.

9.2 Variables les plus importantes

L’importance des variables de Random Forest indique dans quelle mesure chaque variable a contribué à réduire l’impureté des arbres. Les cinq variables les plus importantes cumulent 33,52 % de l’importance totale, le reste étant réparti sur de nombreuses variables.

Variable	Importance	Signification (documentation du dataset)
`ct_state_ttl`	10,09 %	Compteur associé à l’état de la connexion selon des plages de valeurs de TTL source et destination
`sttl`	7,97 %	TTL (durée de vie des paquets IP) côté source vers destination
`sload`	6,53 %	Débit de la source, en bits par seconde
`rate`	4,62 %	Taux de trafic de la connexion
`dload`	4,31 %	Débit de la destination, en bits par seconde

Tableau 6 — Les cinq variables les plus importantes de Random Forest.

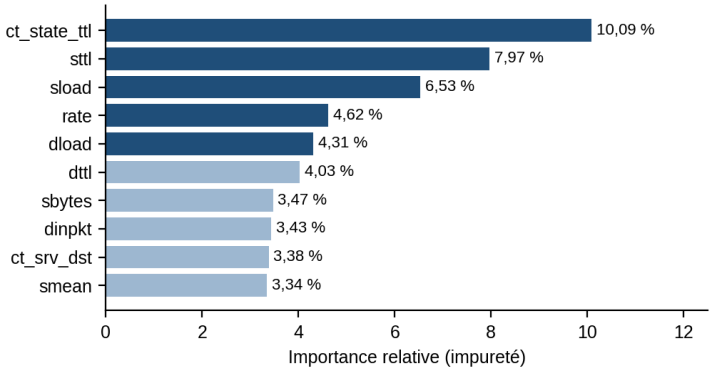


Figure 4 — Dix variables les plus importantes de Random Forest (les cinq premières en bleu foncé).

Ces variables concernent l’état des connexions, les valeurs TTL et les débits de trafic, ce qui est cohérent avec la nature de la tâche. Il faut cependant lire ces résultats avec prudence. **Importance n’est pas causalité** : une variable importante aide le modèle à séparer les classes dans ce dataset, sans prouver qu’elle explique la nature d’une attaque. L’importance par impureté est calculée sur les données d’entraînement et favorise les variables à nombreuses valeurs distinctes ; la documentation de scikit-learn recommande en complément l’importance par permutation sur données de test [8]. Enfin, les valeurs TTL peuvent refléter des particularités de l’environnement de génération du dataset : c’est une hypothèse de travail, non testée ici, qui souligne l’intérêt d’une validation sur d’autres données. Ces importances sont globales et n’expliquent pas à elles seules une alerte individuelle.

10. Simulation de dérive des données et surveillance

10.1 Principe et protocole

La **dérive des données** (*data drift*) désigne l’évolution de la distribution des données reçues par rapport à celles de l’entraînement, par exemple lorsque le trafic d’un réseau change avec le temps. Pour observer la sensibilité de Random Forest, le notebook crée un scénario volontairement artificiel : une copie du jeu de test est modifiée en multipliant par 1,20 (+20 %) les valeurs de trois variables numériques, `dur` (durée), `spkts` et `dpkts` (nombres de paquets source et destination). Ces trois variables sont les trois premières colonnes numériques du fichier, sélectionnées par leur ordre et non pour une raison opérationnelle. Le modèle déjà entraîné est évalué sur cette copie, sans réentraînement ; le jeu de test original reste intact.

Scénario	Accuracy	Rappel attaques	F1 attaques	ROC-AUC
Test original	89,29 %	97,51 %	90,93 %	98,07 %
Test avec dérive synthétique	88,62 %	97,89 %	90,45 %	98,15 %
Écart (points de pourcentage)	−0,67	+0,38	−0,48	+0,08

Tableau 7 — Random Forest : effet de la dérive synthétique (les écarts sont calculés à partir des valeurs du notebook).

10.2 Interprétation et limites

Dans ce scénario, les indicateurs varient peu : l'accuracy et le F1-score baissent légèrement, le rappel et le ROC-AUC augmentent légèrement. Un rappel en hausse accompagné d'un F1-score en baisse implique mécaniquement une précision en baisse : le décalage a probablement fait classer davantage de flux comme attaques, ce qui est un déplacement du compromis entre fausses alertes et détections, non une amélioration. La précision, le nombre de faux positifs et de faux négatifs sous dérive n'ont pas été enregistrés dans le notebook ; ils devront l'être.

> **Cette simulation est synthétique.** Elle teste un seul modèle, un seul scénario et un seul coefficient sur trois variables. Elle n'introduit aucune nouvelle attaque : elle ne constitue ni un test d'attaques inconnues (*zero-day*), ni une preuve de robustesse, ni une simulation réaliste de l'évolution d'un réseau.

10.3 Proposition de surveillance

Pour détecter une dérive réelle en exploitation, le projet propose le suivi suivant. Les seuils ci-dessous sont des points de départ à valider avec le SOC, non des seuils universels, et n'ont pas été testés.

Élément surveillé	Fréquence proposée	Déclencheur de revue
Volume d'alertes et distribution des scores	Quotidienne	Augmentation brutale du volume d'alertes
Dérive des variables d'entrée et des catégories de trafic	Hebdomadaire	Dérive forte d'une variable clé
Taux de FP et de FN sur échantillon labellisé	Mensuelle	Baisse du rappel de plus de 5 points par rapport à la référence
Temps de traitement et erreurs du pipeline	Continue	Anomalie de fonctionnement

Tableau 8 — Plan de surveillance proposé.

11. Limites, gouvernance et recommandations

11.1 Limites

- **Dataset :** UNSW-NB15 est issu d'un environnement de laboratoire et mêle trafic normal réel et attaques synthétiques [2] ; il ne représente pas nécessairement la diversité d'un trafic réel. La proportion d'attaques diffère entre entraînement et test (68,06 % contre 55,06 %). L'étiquette binaire masque les catégories d'attaque.
- **Modèles :** Random Forest dépend de la qualité des étiquettes et n'a pas été réglé (hyperparamètres par défaut). Isolation Forest n'a été testé qu'avec une configuration, au seuil automatique.
- **Seuils :** les prédictions utilisent les seuils par défaut (0,5 pour Random Forest, seuil automatique pour Isolation Forest). Aucun jeu de validation n'a servi à choisir un point de fonctionnement.
- **Conditions expérimentales :** exécution unique, graine fixée, pas de validation croisée ni d'intervalle de confiance ; les durées de prédiction et les ressources nécessaires n'ont pas été mesurées ; les données sont des caractéristiques de flux déjà calculées, sans chaîne de capture du trafic.
- **Interprétabilité et dérive :** l'importance des variables ne prouve ni causalité ni stabilité dans un autre environnement ; la simulation de dérive est limitée (section 10).

11.2 Gouvernance et supervision humaine

Le modèle assiste l'analyste ; il ne doit pas décider seul. Avec 20,78 % de faux positifs sur le trafic normal et 2,49 % de faux négatifs, une action automatique (blocage, sanction, coupure d'un service) fondée sur la seule sortie du modèle pourrait pénaliser une activité légitime ou, à l'inverse, donner une fausse assurance. Les alertes doivent donc être examinées par un analyste SOC avant toute décision importante, selon les procédures de l'organisation. Cette démarche s'inscrit dans les approches de gestion des risques des systèmes d'IA, comme le cadre NIST AI RMF [11]. Concrètement :

- l’analyste reste responsable de l’investigation et de la qualification de l’alerte ; l’alerte lui présente le score, les principales variables contributives, le contexte réseau et les éléments à vérifier ;
- l’équipe qui déploie le modèle documente les données, les versions, les seuils, les tests et les changements, et journalise les alertes et les décisions des analystes ;
- les données réseau pouvant contenir des informations sensibles, leur accès est restreint, leur durée de conservation définie et leur usage conforme au cadre légal applicable.

11.3 Recommandations

Prochaines étapes réalistes (consolidation de l’expérimentation).

- **Régler les seuils** sur un jeu de validation indépendant, en observant le compromis précision–rappel, et en choisissant le point de fonctionnement avec le SOC selon le coût des FP et des FN ; le jeu de test reste réservé à l’évaluation finale. Cela vaut pour Random Forest comme pour Isolation Forest.
- **Réduire les faux positifs :** étudier les flux concernés (Tableau 5), prioriser les alertes par score plutôt que par décision binaire, et envisager des règles complémentaires ou des listes d’exceptions validées.
- **Compléter l’évaluation :** rappel par catégorie d’attaque, importance par permutation [8], validation croisée, mesure du temps de prédiction et des ressources.
- **Valider sur des données indépendantes et représentatives**, idéalement issues du réseau cible, avant toute conclusion opérationnelle.

Vision à plus long terme (prototype opérationnel).

- **Surveiller la dérive** selon le plan du Tableau 8, en enregistrant aussi précision, FP et FN sous dérive.
- **Réentraîner de manière contrôlée :** uniquement après validation d’un changement documenté, avec versionnage des modèles et possibilité de retour arrière.
- Mettre en place la collecte du trafic, la journalisation des alertes, la supervision humaine et les procédures de mise à jour du modèle.

12. Conclusion

Ce projet a comparé deux approches de Machine Learning pour la détection d’intrusions réseau sur UNSW-NB15. Dans les configurations testées, Random Forest obtient les meilleurs résultats (accuracy de 89,29 %, rappel des attaques de 97,51 %, F1-score de 90,93 %, ROC-AUC de 98,07 %), tandis qu’Isolation Forest, avec son seuil automatique, ne détecte que 33,08 % des attaques. Ce rappel élevé de Random Forest s’accompagne de 7 687 fausses alertes, ce qui rend la supervision humaine et la maîtrise des seuils indispensables.

L’analyse des erreurs, l’examen des variables importantes et la simulation de dérive enrichissent l’interprétation sans la transformer en preuve : l’importance statistique n’est pas une causalité et la dérive simulée n’est pas un test d’attaques inconnues. Les résultats constituent une base expérimentale solide, non une validation de déploiement en production. Les prochaines étapes (réglage de seuils sur validation indépendante, évaluation sur données représentatives, surveillance de la dérive, réentraînement contrôlé) permettraient de passer de cette expérimentation à un prototype opérationnel.

Références bibliographiques

[1] Moustafa, N. et Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). *2015 Military Communications and Information Systems Conference (MilCIS)*, IEEE, p. 1–6. DOI : 10.1109/MilCIS.2015.7348942.

[2] Cyber Range Lab, UNSW Canberra. *The UNSW-NB15 Dataset*. https://research.unsw.edu.au/projects/unsw-nb15-dataset (consulté le 31 octobre 2026).

[3] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. DOI : 10.1023/A:1010933404324.

[4] Liu, F. T., Ting, K. M. et Zhou, Z.-H. (2008). Isolation Forest. *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, p. 413–422. DOI : 10.1109/ICDM.2008.17.

[5] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[6] scikit-learn, documentation officielle : *RandomForestClassifier*. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[7] scikit-learn, documentation officielle : *IsolationForest*. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

[8] scikit-learn, documentation officielle : *Permutation Importance vs Random Forest Feature Importance (MDI)*. https://scikit-learn.org/stable/auto_examples/inspection/plot_permutation_importance.html

[9] scikit-learn, guide utilisateur : *Metrics and scoring: quantifying the quality of predictions*. https://scikit-learn.org/stable/modules/model_evaluation.html

[10] Bibliothèques utilisées : pandas (https://pandas.pydata.org), NumPy (https://numpy.org), Matplotlib (https://matplotlib.org).

[11] National Institute of Standards and Technology (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1. DOI : 10.6028/NIST.AI.100-1.

Dépôt GitHub du projet

Le code source, le notebook Jupyter, les résultats expérimentaux et la documentation du projet APSI-NE – Détection d’intrusions sont disponibles sur GitHub :

https://github.com/seydoudangandeabdoulaziz/APSI-NE-Detection-Intrusions